Computers and Mathematics with Applications 63 (2012) 1519–1535

Contents lists available at SciVerse ScienceDirect

Computers and Mathematics with Applications

journal homepage: www.elsevier.com/locate/camwa

Dempster–Shafer evidential theory for the automated selection of parameters for Talbot's method contours and application to matrix exponentiation☆

Patrick O. Kano [a,*], Moysey Brio [a,b], Paul Dostert [c], Jon Cain [b]

[a] Acunum Algorithms and Simulations LLC, 451 Sierra Street, Fernley, NV 89408, United States
[b] Department of Mathematics, University of Arizona, 617 N. Santa Rita Ave., Tucson, AZ 85721, United States
[c] Coker College, E. College Avenue, Hartsville, SC 29550, United States

ARTICLE INFO

Article history:
Received 21 February 2011
Received in revised form 30 November 2011
Accepted 15 March 2012

Keywords:
Numerical Laplace transform inversion
Talbot's method
Dempster–Shafer evidential theory
Matrix exponential
Random matrices

ABSTRACT

In this paper, the Dempster–Shafer theory of evidential reasoning is applied to the problem of optimal contour parameters selection in Talbot's method for the numerical inversion of the Laplace transform. The fundamental concept is the discrimination between rules for the parameters that define the shape of the contour based on the features of the function to invert. To demonstrate the approach, it is applied to the computation of the matrix exponential via numerical inversion of the corresponding resolvent matrix. Training for the Dempster–Shafer approach is performed on random matrices. The algorithms presented have been implemented in MATLAB. The approximated exponentials from the algorithm are compared with those from the rational approximation for the matrix exponential returned by the MATLAB *expm* function.

© 2012 Elsevier Ltd. All rights reserved.

1. Introduction

The numerical inversion of the Laplace transform is a problem which has generated a considerable body of mathematical research due to the practical merit of its solution and its inherent ill-posedness. The method introduced by A. Talbot is one of the more widely known numerical inversion procedures. The fundamental concept of Talbot's method [1] is the deformation of the standard Bromwich contour for analytic Laplace transform inversion to one that allows for a more rapid convergence of a quadrature estimate of the complex inversion integral. A limitation to the practical implementation of Talbot's method is the need to appropriately choose the parameters which define the deformed contour. In this paper, we introduce an approach to select the parameters for Talbot's contours based on the application of Dempster–Shafer evidential theory [2]. To be concise, we refer to this combination of mathematical methods as a Dempster–Shafer–Talbot [DST] numerical inversion approach.

The core concept is the use of previous algorithm training on adjacent problems to select appropriate complex contour defining parameters in new situations. In this work, we show how one may choose features, define mass functions, and apply the Dempster–Shafer fusion algorithm to the selection of an optimal rule for the contour parameters from a given set of rules. To illustrate and to allow for a concrete selection of features, the approach is applied to the numerical inversion of the resolvent matrix of a complex matrix. The time domain inverse is the matrix exponential. This illustration choice allows for Dempster–Shafer algorithm training through comparisons with matrix exponentials computed by means of a standard rational approximation. It also provides for a quantitative analysis of the efficacy of our approach.

☆ The paper has been evaluated according to old Aims and Scope of the journal.

* Corresponding author. Tel.: +1 520 360 7268.
E-mail addresses: pkano@acunum.com (P.O. Kano), brio@acunum.com (M. Brio), pdostert@coker.edu (P. Dostert), cain1@email.arizona.edu (J. Cain).

0898-1221/$ – see front matter © 2012 Elsevier Ltd. All rights reserved.
doi:10.1016/j.camwa.2012.03.041

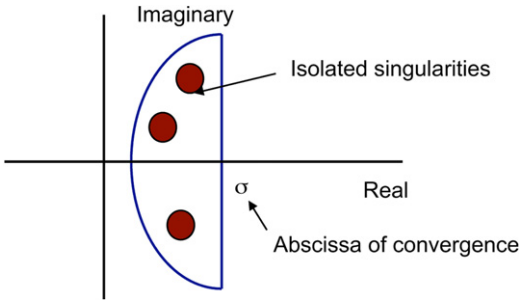


Fig. 1. The Bromwich contour Γ with the abscissa of convergence denoted by σ.

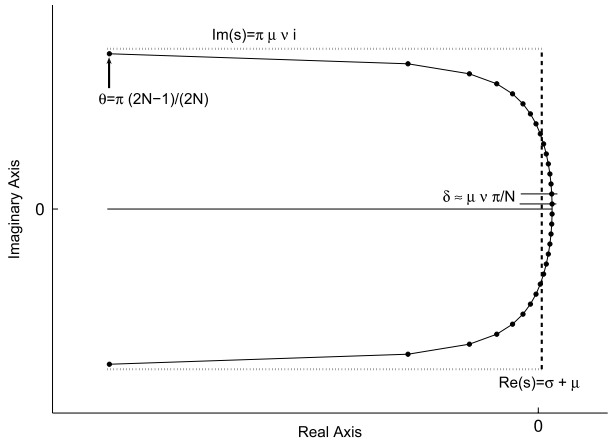


Fig. 2. The Talbot contour with parameters σ, μ, and ν. σ shifts the contour along the real axis. μ and ν affect the contour's width. Numerical integration is performed using the values of $F(s)$ at $\delta\theta$ separated points on the contour.

The paper is divided into four sections. Following this introduction, the second section involves a description of the mathematical approach in sufficient detail for replication. This includes a description of Talbot's method, the parameters which define the Talbot contour, and a description of the Dempster–Shafer theory as it is applied to the discrimination between parameter selection rules. A rule can be as simple as a set of constant parameters or a choice of functions that relate the features of $F(s)$ to values for the contour parameters. This is followed in the third section by a discussion of the matrix exponential that focuses on its relationship to the Laplace transform. We then apply the DST procedure to the inversion of the resolvent matrix and present results from an efficacy analysis. Matrices with qualitatively different distributions of their complex eigenvalues are used for the analysis. The method by which training of the DST approach to matrix exponentiation is performed using random matrices with uniformly distributed elements is also discussed. The last concluding section summarizes the work, discusses strengths and weakness, and provides suggestions for further research.

2. Algorithm description

This section provides the mathematical theory for the Dempster–Shafer–Talbot approach to numerical Laplace transform inversion. Talbot's method and the contour parameters that one must define when using this inversion procedure are discussed. A general description of the Dempster–Shafer evidential theory as it applies to Talbot's method is also formulated. The details of the application of the DST approach to matrix exponentiation are provided in Section 3.

2.1. Talbot's method

The forward transform and its inverse can be defined through the equations

$$F(s) \equiv \mathscr{L}(f(t)) = \int_0^\infty e^{-st} f(t)dt \tag{1}$$

$$F(s) : \mathbb{C} \to \mathbb{C}^n$$

$$f(t) \equiv \mathscr{L}^{-1}(F(s)) = \frac{1}{2\pi i}\int_\Gamma e^{st} F(s)ds$$

$$f(t) : [0, \infty) \to \mathbb{C}^n.$$

Table 1
Features of matrix A.

Feature	Expression	Motivation
Trace real part	$\mathscr{R}(\text{trace}(A)) = \mathscr{R}\left(\sum_{j=1}^{J} A_{jj}\right)$	The trace is the sum of the eigenvalues of A, $\sum_j \lambda_j(A)$.
One norm	$\|A\|_1 = \max_{1\le j\le J}\left[\sum_{k=1}^{K} \|A_{jk}\|\right]$	The maximum absolute column sum is quickly computed.
Frobenius norm	$\|A\|_F = \sqrt{\text{trace}(A^*A)}$	For Hermitian matrices, this equals the spectral radius $\rho(A)$.
Infinity norm	$\|A\|_\infty = \max_{1\le k\le K}\left[\sum_{j=1}^{J} \|A_{jk}\|\right]$	The maximum absolute row sum is quickly computed.

Table 2
Contour parameter selection rules for exponentiation of the complex scaled square matrix A. The matrix norm is the Frobenius norm.

Class	σ	ν	μ
1	0	1	1
2	$\mathscr{R}(\text{trace}(A))$	1	1
3	0	$\frac{1}{\|A\|}$	$\|A\|$
4	$\|A\|$	$\frac{1}{\|A\|}$	$\|A\|$
5	$\|A\|$	1	1
6	$\mathscr{R}(\text{trace}(A))$	$\|A\|$	$\sqrt{\nu}$
7	$\mathscr{R}(\text{trace}(A))$	$\|A\|$	ν
8	$\mathscr{R}(\text{trace}(A))$	$\|A\|$	ν^2
9	$\mathscr{R}(\text{trace}(A))$	$\|A\|$	ν^3

Table 3
Training on square matrices with randomly selected complex elements.

Parameter	Value
Matrix dimensions	2, 3, . . . , 100
Number matrices per dimension	1500
Element real part bounds	[−100, 100]
Element imaginary part bounds	[−100, 100]
Random seed	1234

Table 4
Feature statistics for the scaled matrices in the training set.

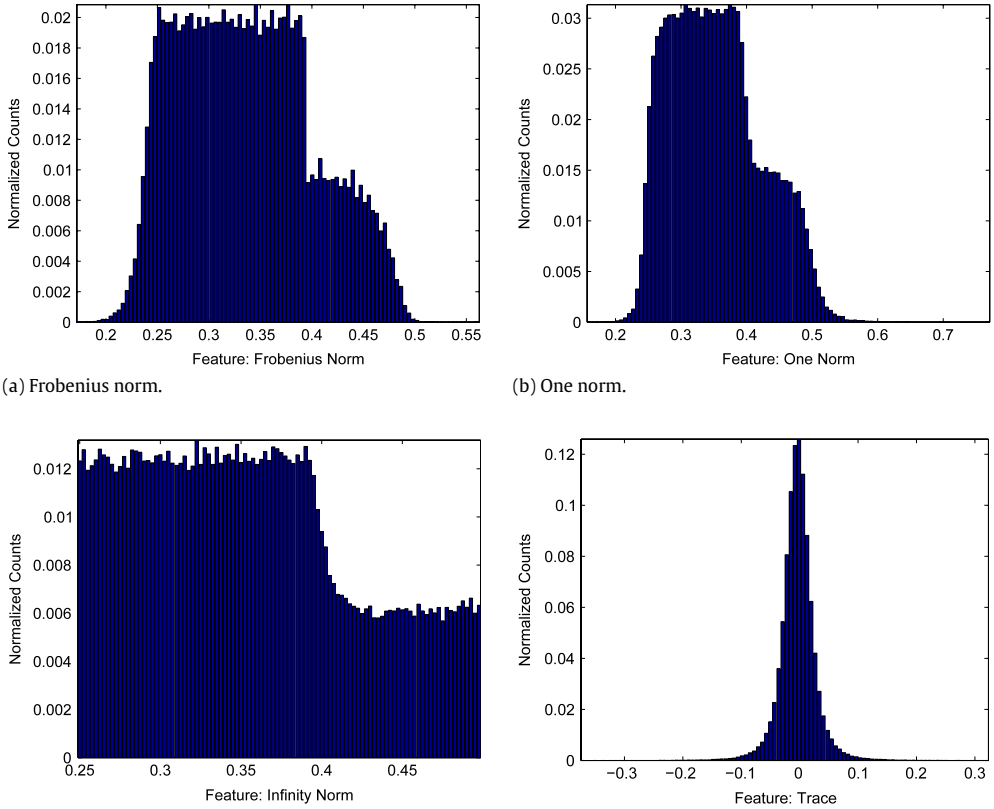
Feature	Minimum	Mean	Maximum
1-norm	0.15929	0.35668	0.77462
∞-norm	0.25	0.35648	0.5
Frobenius norm	0.17332	0.33909	0.5649
Trace real part	−0.37169	$-3.0125 \cdot 10^{-5}$	0.32733

For many applications, one considers $F(s)$ and $f(t)$ to be scalar functions and $f(t)$ to be real valued. They can however be complex and multivalued as in the case of matrix exponentiation. The forward transform $f(t) \rightarrow F(s)$ exists for an exponentially bounded $f(t)$ and is typically straightforward to compute. The inverse Laplace transform however is defined through a complex integration along the contour Γ. The Bromwich contour, Fig. 1, is the most common choice for Γ. It passes through the real axis at a value that is greater than the largest real part of the singularities of $F(s)$. This minimum real value is known as the abscissa of convergence. For simple expressions for $F(s)$, analytic methods for integration along this complex contour are applicable. For example, for isolated poles, one finds from Cauchy's residue theorem that $f(t)$ is simply the sum of the residues of $F(s)$. For many real world applications however, the complexity of the Laplace space function prohibits direct analytic inversion. In this case, a numerical inversion can be a viable alternative.

The numerical inversion of the Laplace transform is a well known example of an inherently ill-posed problem [3,4]. Inaccuracies due to finite precision and numerical approximations are amplified by the exponential term and thus can be significant for even modest inversion times. There exist a number of regularization procedures for the numerical inversion such as the Weeks method [5] and Post's formula [6]. Each has application to the inversion of certain classes of functions. Works summarizing the various methods are [7,8].

This paper focuses on the method introduced by Talbot [1]. Talbot's method replaces the Bromwich contour with one that opens towards the negative real axis, see Fig. 2. The deformed contour is described by the following Eqs. (2).

$$s(\theta) = \sigma + \mu\theta(i\nu + \cot(\theta)) \tag{2}$$
$$\frac{ds}{d\theta} = i\nu\mu + \mu\left[\cot(\theta) - \theta\csc^2(\theta)\right]$$



(a) Frobenius norm.

(b) One norm.

(c) Infinity norm.

(d) Trace real part.

Fig. 3. Histograms of the features from the training set matrices.

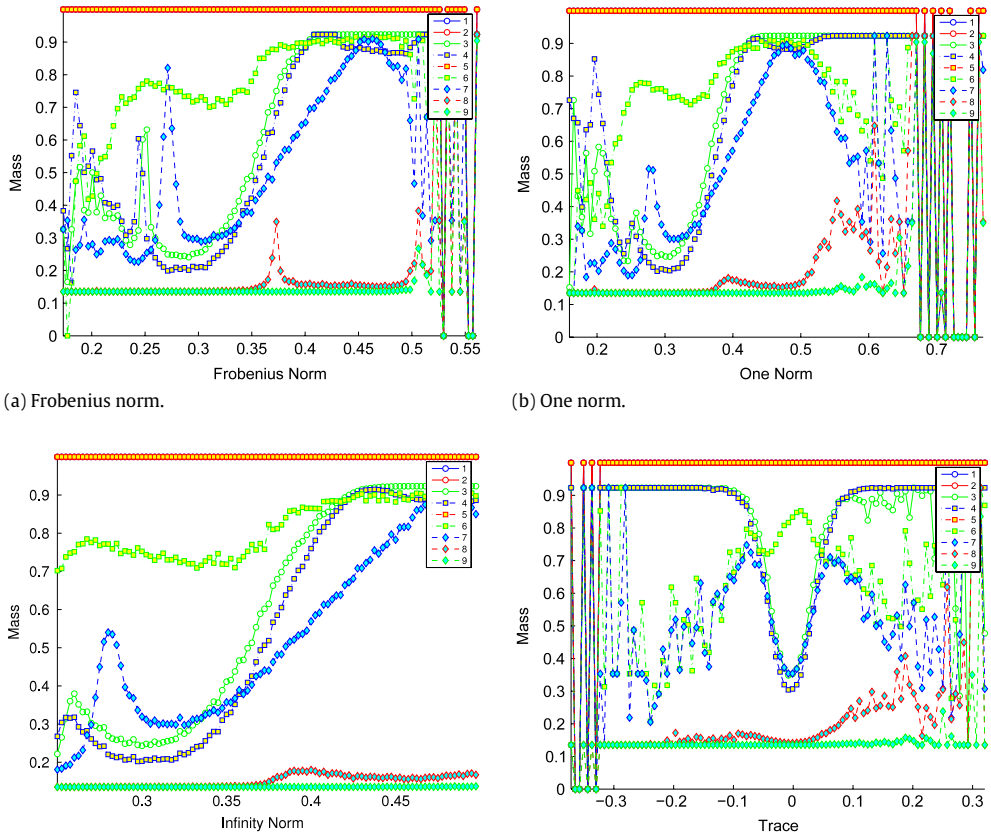
$$\sigma \in (-\infty, \infty)$$
$$\mu \in [0, \infty)$$
$$\nu \in [0, \infty)$$
$$\theta \in (-\pi, \pi).$$

Cauchy's integral theorem allows this deformation so long as the contour does not pass through singularities. Talbot's method requires that $F(s) \to 0$ as $|s| \to \infty$ and is clearly limited to Laplace space functions whose singularities have bounded imaginary components. The method inherently regularizes through damping highly oscillatory terms and thus may be inappropriate for purely conservative problems. The damping however is a stabilizing procedure since it is often these high oscillatory terms which provide the largest contribution to the error when one performs numerical quadrature of the Laplace inversion integral.

When discretized for numerical evaluation, the Talbot contour integral can be estimated by a midpoint rule

$$f_N(t) = \frac{1}{2iN} \sum_{k=-N}^{N-1} e^{s(\theta_k)t} \frac{ds}{d\theta}(\theta_k) F(s_k) \tag{3}$$
$$\theta_k = (2k+1)\frac{\pi}{2N}$$
$$k = -N, \ldots, N-1$$
$$\delta\theta = \frac{\pi}{N}.$$

The midpoint θ parameterized points are those plotted in the example contour in Fig. 2. The Talbot contour extends towards negative real infinity at $\theta = \pm\pi$. The figure however truncates at the last midpoint values where the function $F(s)$ is evaluated for the numerical inversion. Note that this contour does have a finite width that is defined by the contour parameters μ and ν. The parameter σ is a shift of the location at which the contour and the real axis intersect. This choice for the discretization also avoids evaluating the parameterized expression $s(\theta)$ at the removable singularity at $\theta = 0$. Rather, $s(\theta)$ is evaluated at the midpoints $\theta_{k=0} = \frac{\pi}{2N}$ and $\theta_{k=-1} = \frac{-\pi}{2N}$.



(a) Frobenius norm.

(b) One norm.

(c) Infinity norm.

(d) Trace real part.

Fig. 4. Mass functions versus the feature values from the training activity.

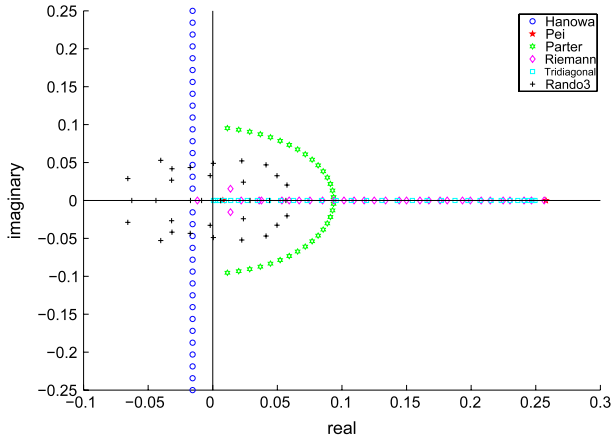


Fig. 5. Eigenvalues for the scaled test case matrices.

In both Talbot's original 1979 paper and in subsequent work [9–11,4], a considerable effort has been made to derive rules for the proper selection of the contour parameters. These approaches have focused on the known singularities or the form for the functions to invert. For even simple scalar functions, this direct parameter selection approach places a burden on a general Talbot method user to have considerable knowledge about the Laplace space function $F(s)$. In this paper, instead of a deterministic parameter selection rule, we have adopted Dempster–Shafer evidential theory to automatically decide upon a most accurate rule from a set of possible ones. This approach has been partially motivated by considering the quote from Talbot that his own parameter selection strategy has been "based on an intensive study of a large number of experimental results using many different types of function $F(s)$" [1]. Given Talbot's own selection of contour parameters as being based on "experimental results", this application of evidential theory appears appropriate.

Table 5
Feature values versus the size of the square test matrices.

Feature	16	32	64	128
(a) Rando3				
1-norm	0.4063	0.3906	0.3984	0.3906
∞-norm	0.4063	0.4219	0.3906	0.3828
Frobenius norm	0.3989	0.4026	0.4047	0.4075
Trace real part	0.1250	0.01563	0.03906	0.007813
(b) Hanowa				
1-norm	0.2813	0.2656	0.2578	0.2539
∞-norm	0.2813	0.2656	0.2578	0.2539
Frobenius norm	0.6435	0.8592	1.1834	1.6527
Trace real part	−0.5	−0.5	−0.5	−0.5
(c) Pei				
1-norm	0.2656	0.2573	0.2539	0.2520
∞-norm	0.2656	0.2578	0.2539	0.2520
Frobenius norm	0.2724	0.2614	0.2558	0.2527
Trace real part	0.5	0.5	0.5	0.5
(d) Parter				
1-norm	0.2527	0.2960	0.3393	0.3827
∞-norm	0.2527	0.2960	0.3393	0.3827
Frobenius norm	0.3782	0.5439	0.7765	1.1038
Trace real part	1.0	2.0	4.0	8.0
(e) Riemann				
1-norm	0.5938	0.6953	0.8164	0.8926
∞-norm	0.4844	0.4922	0.4962	0.4980
Frobenius norm	0.6827	0.9333	1.2922	1.8098
Trace real part	2.125	4.125	8.125	16.125
(f) Tridiagonal				
1-norm	0.25	0.25	0.25	0.25
∞-norm	0.25	0.25	0.25	0.25
Frobenius norm	0.6060	0.8615	1.2216	1.7298
Trace real part	2.0	4.0	8.0	16.0

2.2. Dempster–Shafer evidential theory

The Dempster–Shafer evidential theory is a well known method in decision science [12,13,2,14,15]. Briefly stated, it is a mathematical theory that defines how one can weight and fuse multiple pieces of evidence to arrive at a single decision when the pieces have varying degrees of relevance.

The basic concept is the use of belief functions to assign confidence to single or combinations of decision states. To combine belief from different sources, one uses the Dempster–Shafer fusion rule. A decision can then be made based on the single fused belief on each decision state.

Dempster–Shafer theory is an alternative to classical Bayesian probability theory. It provides for bounds of the confidence in a decision state which include the classical probability. It is also different from standard probability in that it allows for confidence to be assigned to a set of states instead of to each individual state.

2.2.1. Fundamentals

The fundamental quantities in the theory are:

sources of evidence The functions or sensor data that provide the raw information.
decision states (Θ) The single possible states between which one must decide.
frame of discernment (2^{Θ}) The power set of possible single decision states.
features Discriminating information that is extracted from the raw source data that allows one to choose between the states.
basic probability assignment (m) The mass assigned to a decision state or collection of decision states based on the feature values.
belief and plausibility $b(\Theta)$, $pl(\Theta)$ The final decision metrics from the fusion of the masses for different features on the decision states.

Application of Dempster–Shafer theory occurs in two phases. In the first *training* phase, one identifies the decision states and discriminating features from tests data sets. Mass functions for each class and feature pair are also assigned. Then, in

Table 6
Minimum relative error versus number of contour points [row] and the matrix size [column].

	16	32	64	128
(a) Rando3				
16	[5] 2.57e−3	[5] 1.11e−2	[2] 4.48e−2	[1] 1.78e−1
32	[5] 1.02e−5	[5] 4.21e−5	[2] 1.73e−4	[1] 6.90e−4
64	[5] 1.48e−8	[5] 6.24e−8	[2] 2.54e−7	[1] 1.02e−6
128	[5] 1.00e−12	[5] 4.62e−12	[5] 1.70e−10	[1] 1.40e−10
(b) Hanowa				
16	[1] 1.01e−3	[1] 2.87e−3	[1] 8.04e−3	[1] 2.27e−2
32	[2] 3.40e−6	[1] 1.10e−5	[3] 1.96e−5	[1] 8.71e−5
64	[2] 5.10e−9	[1] 1.62e−8	[9] 6.09e−9	[9] 5.83e−12
128	[2] 2.89e−13	[1] 1.30e−12	[9] 1.25e−12	[8] 1.28e−12
(c) Pei				
16	[2] 5.33e−3	[2] 2.12e−2	[2] 8.47e−2	[2] 3.38e−1
32	[1] 2.10e−5	[2] 8.39e−5	[2] 3.35e−4	[2] 1.34e−3
64	[2] 3.06e−8	[2] 1.22e−7	[2] 4.87e−7	[2] 1.95e−6
128	[1] 2.14e−12	[1] 8.56e−12	[1] 2.27e−11	[2] 6.23e−10
(d) Parter				
16	[2] 2.59e−3	[5] 4.26e−3	[5] 4.78e−3	[1] 2.25e−2
32	[2] 6.32e−6	[5] 1.88e−5	[5] 2.97e−5	[5] 4.27e−5
64	[2] 9.18e−9	[5] 2.58e−8	[5] 3.46e−8	[3] 7.79e−8
128	[2] 9.25e−13	[5] 1.82e−12	[5] 3.81e−12	[1] 1.18e−11
(e) Riemann				
16	[5] 5.30e−3	[4] 5.97e−3	[3] 3.96e−2	–
32	[5] 2.03e−5	[5] 6.34e−5	[5] 1.15e−4	–
64	[5] 2.98e−8	[5] 7.37e−8	[3] 1.23e−7	–
128	[5] 1.87e−12	[5] 3.42e−12	[3] 1.29e−11	–
(f) Tridiagonal				
16	[5] 1.35e−3	[5] 2.10	[4] 33.85	[4] 97.84
32	[1] 4.93e−6	[4] 2.09	[9] 33.85	[3] 97.84
64	[1] 7.15e−9	[7] 2.09	[3] 33.85	[8] 97.84
128	[2] 1.33e−10	[6] 2.10	[6] 33.85	[6] 97.84

the second *application* phase, feature values are obtained from the raw data and the mass functions are used to weight each decision class.

Again, in contrast to standard Bayesian probability, in Dempster–Shafer theory the basic probability assignments are made to elements of the power set 2^{Θ} instead of only to single events. That is, a belief mass function $m : 2^{\Theta} \to [0, 1]$ has the two properties: $m(\emptyset) = 0$ and $\sum_{A \in 2^{\Theta}} m(A) = 1$ where the sum is over the entire power set.

Typically, one has multiple mass functions which correspond to different features. The mass functions are defined through engineering judgment or by training the algorithm prior to its employment in a real system.

The masses from multiple features can be fused into a single mass using the Dempster–Shafer fusion rule:

$$m^{1,2}(C) = \frac{\sum\limits_{A \cap B = C} m^1(A) m^2(B)}{1 - \sum\limits_{A \cap B = \emptyset} m^1(A) m^2(B)}. \tag{4}$$

This formula states that the fused mass assigned to a power set element C is the ratio of the summed masses from sets A and B supporting the hypothesis to the sum of the masses from A and B in conflict with C. The superscript corresponds to features 1 and 2. For multiple features, the order of the fusion is unimportant; the operation commutes. The fusion computation also entails a large number of combination rule evaluations. A binary matrix representation of the power set can be used to implement efficiently the set intersection operations.

With the fused mass, one computes the belief $b(A)$

$$b(A) = \sum_{B \subseteq A} m(B) \tag{5}$$

which has the properties

1. $b(\emptyset) = 0$
2. $b(\Theta) = 1$
3. $b(A \cup B) = b(A) + b(B)$ if $A, B \subseteq \Theta$ and $b(A \cap B) = \emptyset$

Table 7
Difference between the contour 5 relative error and the minimum relative error.

	16	32	64	128
(a) Rando3				
16	[5] 0.0	[5] 0.0	[2] 4.08e−2	[1] 2.54e−2
32	[5] 0.0	[5] 0.0	[2] 4.74e−5	[1] 3.41e−6
64	[5] 0.0	[5] 0.0	[2] 1.50e−7	[1] 6.69e−8
128	[5] 0.0	[5] 0.0	[5] 0.0	[1] 8.15e−11
(b) Hanowa				
16	[1] 4.29e−4	[1] 2.21e−2	[1] 1.04e0	[1] 1.65e0
32	[2] 1.22e−7	[1] 7.56e−5	[3] 1.70e−4	[1] 2.55e−3
64	[2] 7.70e−10	[1] 2.39e−7	[9] 6.66e−7	[9] 2.31e−6
128	[2] 1.27e−13	[1] 2.04e−11	[9] 5.89e−11	[8] 2.24e−10
(c) Pei				
16	[2] 2.62e−4	[2] 1.13e−3	[2] 4.66e−3	[2] 1.89e−2
32	[1] 5.77e−7	[2] 2.43e−6	[2] 1.01e−5	[2] 1.38e−3
64	[2] 1.15e−9	[2] 4.94e−9	[2] 2.05e−8	[2] 8.34e−8
128	[1] 2.13e−13	[1] 1.50e−12	[1] 2.12e−12	[2] 6.74-e−12
(d) Parter				
16	[5] 0.0	[5] 0.0	[5] 0.0	[1] 8.80e−2
32	[2] 4.02e−6	[5] 0.0	[5] 0.0	[5] 0.0
64	[2] 5.77e−9	[5] 0.0	[5] 0.0	[3] 6.00e−8
128	[2] 7.11e−14	[5] 0.0	[5] 0.0	[1] 3.86e−12
(e) Riemann				
16	[5] 0.0	[4] 3.78e−3	[3] 7.22e−2	–
32	[5] 0.0	[5] 0.0	[5] 0.0	–
64	[5] 0.0	[5] 0.0	[3] 1.60e−8	–
128	[5] 0.0	[5] 0.0	[3] 7.91e−12	–
(f) Tridiagonal				
16	[5] 0.0	[5] 0.0	[4] 1.14e−3	[4] 1.30e−3
32	[1] 3.54e−7	[4] 7.97e−7	[9] 8.27e−4	[3] 3.94e−3
64	[1] 5.35e−10	[7] 2.30e−5	[3] 2.98e−5	[8] 2.97e−7
128	[2] 4.78e−14	[6] 2.18e−9	[6] 3.17e−9	[6] 1.12e−4

and the plausibility $pl(A)$

$$pl(A) = \sum_{B \cap A \neq \emptyset} m(B). \tag{6}$$

Equivalently, the plausibility of A can be defined as one minus the belief on A, $pl(A) + b(\sim A) = 1$.

The belief and plausibility bracket the standard Bayesian probability $P(A)$ in that $b(A) \leq P(A) \leq pl(A)$. In the special case that the belief and plausibility are equal, one obtains the usual statement in probability that $P(A) + P(\sim A) = 1$. One sees from these definitions that the belief is a measure of direct evidence for a decision state while the plausibility is a measure of how much evidence is not in direct conflict with the decision. A final decision can be made based on the maximum belief or plausibility.

2.2.2. Application to Talbot's method contours

When applied to Talbot's method, the set of decision states (Θ) are rules by which to select the contour parameters (μ, σ, ν). A rule can be as simple as a triplet of constant parameter values or a choice of functions relating the feature values to the parameters. The exact selection for the rules will depend on the form of $F(s)$. Note that this approach to defining parameter selection rules is intentionally 'looser' than those of previous authors. In those works [9–11,4], the authors have used specific rules. Here instead, with algorithm training on a family of $F(s)$ functions, good rules are determined.

The mass functions m are defined by computing Talbot's method on the contours from the rules for functions $F(s)$ with known inverses $f(t)$. m is then defined from the error for the approximate inverse. The features of $F(s)$ are also computed and the mass assigned to a (feature, contour rule) pair.

When confronted with a new $F(s)$, one applies the standard Dempster–Shafer theory approach:

1. *Feature extraction* extract feature values $\{f\}$ from $F(s)$.
2. *Mass assignments* assign mass $m_f(A) : A \in 2^{\Theta}$ to each power set element and feature pair.
3. *Dempster–Schafer fusion* fuse the mass over the features for each power set element via the Dempster–Shafer rule.
4. *Belief and plausibility calculation* compute the belief and plausibility on each decision state.

Table 8
Rando3 matrix: Plausibility versus the relative error with 64 contour points.

Contour	*pl*	δ RE	Rank by *pl*
(a) 16×16			
5	0.16927	0	1
2	0.16927	1.04e−9	3
1	0.16927	1.07e−9	2
6	0.14907	2.49e−5	6
3	0.15328	7.87e−5	5
4	0.15380	9.50e−5	4
7	0.12858	4.46e−4	7
8	0.0950	1.17e1	8
9	0.0912	3.72e1	9
(b) 32×32			
5	0.17429	0.0	1
1	0.17429	9.12e−10	2
2	0.17429	9.83e−10	3
6	0.15913	9.05e−5	4
3	0.14410	2.49e−4	5
4	0.14020	2.85e−4	6
7	0.12761	1.55e−3	7
8	0.093395	6.23e−2	8
9	0.091523	4.92e2	9
(c) 64×64			
2	0.17229	0.0	3
1	0.17229	1.55e−9	2
5	0.17229	1.50e−7	1
6	0.15469	7.96e−4	4
4	0.14362	9.63e−4	6
3	0.14905	1.30e−3	5
7	0.12945	1.36e−2	7
8	0.093704	5.69e−1	8
9	0.091386	2.73e3	9
(d) 128×128			
1	0.17804	0.0	2
2	0.17804	1.04e−10	3
5	0.14031	6.69e−8	1
6	0.13460	1.62e−3	4
3	0.17804	2.90e−3	5
4	0.16049	2.98e−3	6
7	0.12419	2.70e−2	7
8	0.093529	1.30e0	8
9	0.091723	3.98e1	9

Here we have chosen the contour rule as the one with the highest plausibility. The numerical integration is then performed on the Talbot contour that is defined by the parameters from the chosen rule.

3. Algorithm illustration

In this section, we illustrate the use of the Dempster–Shafer–Talbot approach. The example is the calculation of the matrix exponential through the numerical Laplace transform inversion of the resolvent matrix. The general discussion in the previous section is concretely formulated and an efficacy analysis of the method presented.

3.1. Matrix exponentiation and numerical Laplace transform inversion

A natural application of the Talbot method for numerical inverse Laplace transformation is the computation of the matrix exponential. The general definition of a function f of a matrix A is

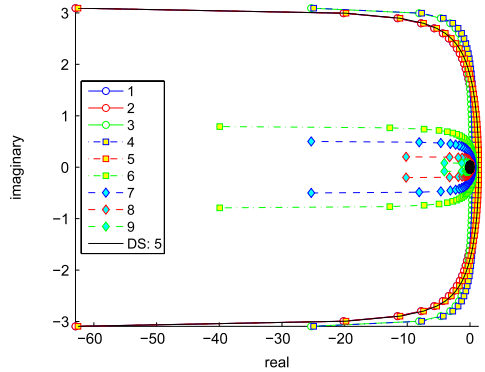
$$f(A) \equiv \frac{1}{2\pi i} \int_{\Gamma} f(s)(sI - A)^{-1} ds. \tag{7}$$

In the particular case that $f(s) = e^{st}$, one obtains a definition for the matrix exponential

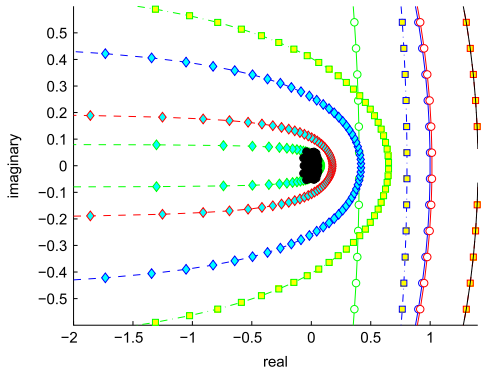
$$e^{At} \equiv \frac{1}{2\pi i} \int_{\Gamma} e^{st}(sI - A)^{-1} ds. \tag{8}$$

Contour	pl	δ RE	Rank by pl
5	0.17429	0.0	1
1	0.17429	9.12e-10	2
2	0.17429	9.83e-10	3
6	0.15913	9.05e-5	4
3	0.14410	2.49e-4	5
4	0.14020	2.85e-4	6
7	0.12761	1.55e-3	7
8	0.093395	6.23e-2	8
9	0.091523	4.92e2	9

(a) Tabulated error.



(b) Discretized Talbot contours.



(c) Eigenvalues and the Talbot contours.

Fig. 6. Rando3 32×32 matrix for 64 contour points.

The Laplace transform space function corresponding to the matrix exponential is thus the resolvent matrix $F(s) = (sI - A)^{-1}$. Indeed, the use of Laplace transform methods for matrix exponentiation is method twelve of the nineteen approaches described in the often cited work by Moler and Van Loan [16]. That text however does not describe the utilization of Talbot's method.

To test the accuracy of the approach, we have adopted the assumption that the rational approximation for the matrix exponential from MATLAB's *expm* function is exact. As will be seen later in this paper, the Talbot method and rational approximation approaches yield competitively accurate results so that this assumption may not be strictly true. However it allows for an initial error analysis, training of the Dempster–Schafer algorithm, and a demonstration of the overall DST approach.

This *expm* algorithm involves computing the Pade' approximation to a scaled input matrix A/m and then squaring the approximation to obtain the desired matrix exponential $e^A = (e^{A/m})^m \approx R_{qp}^m$. It is a combination of methods two and three in the Moler–Van Loan paper [16]. The motivation for the scaling is that it reduces the spread of the eigenvalues and hence controls the round-off error and computing costs of the Pade' approximation. The MATLAB *expm* function uses a $(q = 1, p = 6)$ rational approximation and selects a value for $m = 2^s$ where $s = \max(0, 1 + \log_2(\|A\|_\infty))$. This approximation has the distinct advantage of possessing an analytic expression for its relative error [17]. The expression for s has been chosen in *expm* to ensure that this estimate is less than a given tolerance.

3.2. DST approach to matrix exponentiation

3.2.1. Scaling

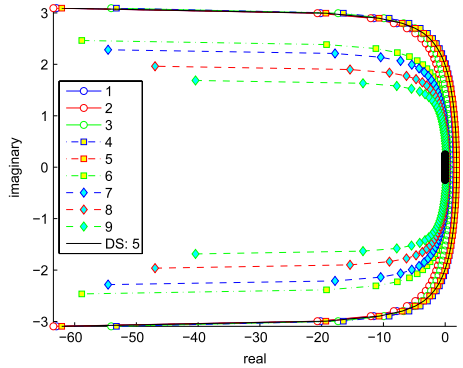
The MATLAB scripts which implement the DST approach return the approximate exponential of a square matrix M with complex elements. As in MATLAB's *expm* function, the scripts do this by computing an approximation of a scaled matrix $A = M/2^s$ where $s = \max(0, 1 + \log_2(\|M\|_\infty))$ and then squaring the result.

3.2.2. Feature extraction

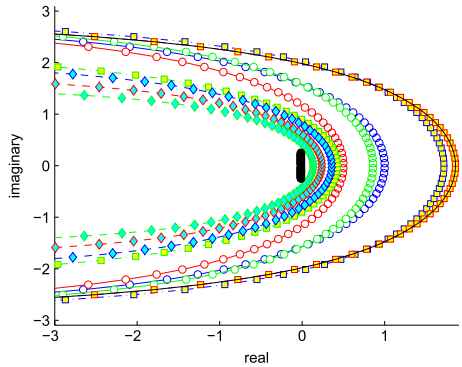
In Table 1 are the four features of the scaled matrix A that are used in the selection of the contour parameter rule. They have been chosen because they can be computed quickly and provide information about the matrix's spectral characteristics.

Contour	pl	δ RE	Rank by pl
1	0.32816	0.0	2
2	0.32816	2.06e-8	3
4	0.23646	1.29e-7	6
5	0.32816	2.39e-7	1
3	0.24252	3.72e-7	5
6	0.29859	2.37e-6	4
7	0.23489	1.10e-5	7
8	0.22113	2.45e-5	8
9	0.22112	2.20e-3	9

(a) Tabulated error.



(b) Discretized Talbot contours.



(c) Eigenvalues and the Talbot contours.

Fig. 7. Hanowa 32 × 32 matrix for 64 contour points.

Note that in practice, the Dempster–Shafer theory usually performs better when features are independent. The matrix norms are related and thus there may be a more discriminating set of features than these. However, our tests have shown that they do provide sufficient discriminating capability.

3.2.3. Hypothesis definition

Listed in Table 2 are the nine contour parameter rules defined for the matrix exponentiation. They are motivated by considering the relationships between the contour parameters and the eigenvalues and norms of a matrix. For example, it is known that accurate inversion requires the eigenvalues to have real part less than the abscissa of convergence. Since the trace is the sum of the eigenvalues and σ shifts the largest real value on the contour, σ was chosen for some contours to be the real part of the trace. Also, since larger μ values correspond to more negative and damping real contour values, we have considered cases where the μ value increases rapidly with the matrix Frobenius norm. These rules were also downselected from a larger set after observing the relative error of the estimate that was obtained from integration on the ruled defined contour. For a detailed study of the dependence of the Talbot contour on its parameters one can see the related work [18].

3.2.4. Mass assignments

The mass functions were obtained by comparing the exponential computed with MATLAB's *expm* Pade approximation and those from all nine Talbot contours. The relative error based on the Frobenius norm

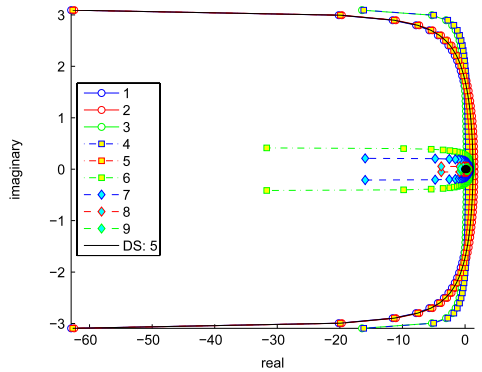
$$E_{\text{relative}} = \frac{\|\alpha - e^M\|_{\text{Frobenius}}}{\|e^M\|_{\text{Frobenius}}}$$

was used to create the mass functions according to the formula

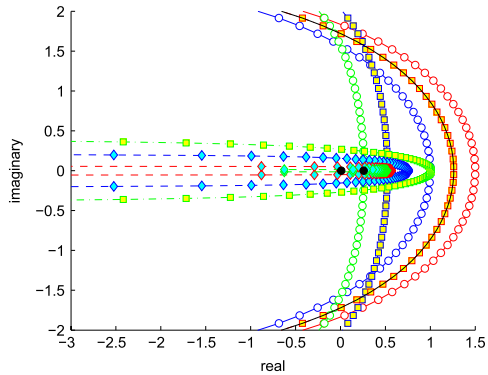
$$\begin{aligned} S &= \{M_n\} = \text{training set of matrices} \\ n &= \text{matrix index} \\ c &= \text{contour index} \\ f_n &= \text{feature value of scaled matrix } A_n \end{aligned} \tag{9}$$

Contour	pl	δ RE	Rank by pl
2	0.23616	0.0	3
1	0.23616	9.64e-10	2
5	0.23616	4.94e-9	1
6	0.20322	3.74e-3	4
4	0.14613	6.24e-3	6
7	0.14245	7.20e-2	7
8	0.13084	7.04e0	8
3	0.15244	3.20e1	5
9	0.13083	3.20e1	9

(a) Tabulated error.



(b) Discretized Talbot contours.



(c) Eigenvalues and the Talbot contours.

Fig. 8. Pei 32 × 32 matrix for 64 contour points.

α_n = approximate nth matrix exponential

E_{nc} = relative error of nth matrix exponential computed on contour c

$\tilde{m}_c(f_n) = \exp(-E_{nc})$.

In practice, the storage and interpolation of the full set $\{\tilde{m}_c(f_n)\}$ is too demanding. Thus, what the DST code actually uses for mass functions are the coarser set $\{m_c(f_n)\}$ obtained by histogramming the feature values. If two test matrices are binned to the same feature value, the maximum of the computed relative errors is taken for $m_c(f_n)$.

When a new matrix is input, the mass assigned to contour c is obtained by interpolating the corresponding set $\{m_c(f_n)\}$ at the feature value f extracted from the matrix. For feature values which occur outside of the bounds defined by the training set matrices, the mass is set to zero. The mass functions on power sets of contours are defined as the sum of the masses on the singleton sets $m_{\{c_j\}}(f) = \sum_j m_{c_j}(f)$. This rule for defining mass on sets as the sum of singleton masses is computationally convenient but not necessary for the DST approach. One can choose other approaches for sets or even combine the singleton masses in a nonuniform manner so long as the basic mass assignment properties are satisfied.

The training to define the mass functions was performed on random matrices. Integration and analysis was performed on each contour for sixty-four midpoints. The matrices are square and have complex elements that are selected from a uniform distribution in a rectangular region of the complex plane. The MATLAB *rand* command is used for the real and imaginary parts. Table 3 summarizes the parameters for the training analysis matrices:

The properties of these random matrices are well known [19]. For example, since each matrix element is sampled from a uniform distribution with zero mean, the mean of the trace is also zero. This is seen in the normalized histogram for the trace real part, Fig. 3. The bounds on the feature values from the training set are reported in Table 4.

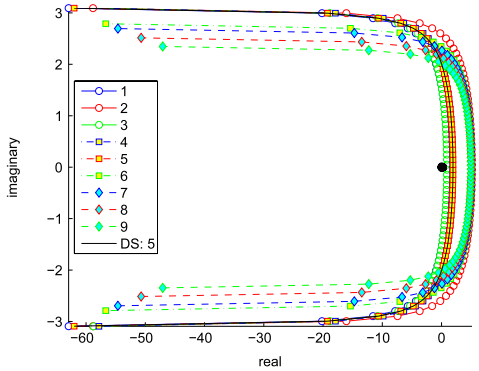
The mass functions corresponding to the four features are shown in Fig. 4. The functions for contours 1, 2, and 5 are consistently closer to 1 than for the other contours. These three contours will therefore be weighted more during Dempster–Shafer evidential reasoning and thus are mostly likely to be selected.

3.2.5. Dempster–Shafer fusion and plausibility calculation

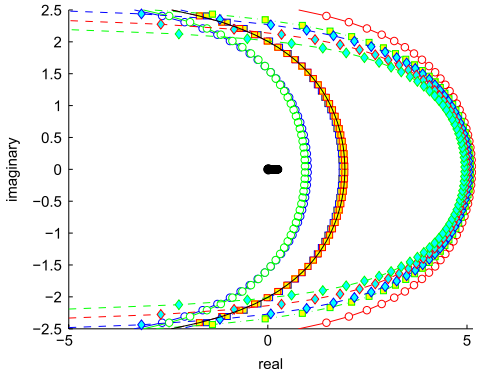
The Dempster–Shafer fusion of the masses from different features for the power sets is accomplished using Eq. (4). A binary representation of the intersections of the 2^9 power sets for the nine possible contours is computed off-line and loaded when the code executes. Determining the set intersections within the code slows considerably the evaluation of

Contour	pl	δ RE	Rank by pl
5	0.28286	0.0	1
4	0.27492	1.91e-8	6
1	0.28286	4.96e-8	2
3	0.27663	9.49e-8	4
2	0.28286	1.53e-3	3
6	0.27540	6.23e-3	5
7	0.27363	1.25e-2	7
8	0.24343	1.31e-2	8
9	0.24025	8.11e-2	9

(a) Tabulated error.



(b) Discretized Talbot contours.



(c) Eigenvalues and the Talbot contours.

Fig. 9. Riemann 32×32 matrix for 64 contour points.

the DS fusion equation. After the masses from the four features have been fused into a single mass, the plausibility pl on single contour rules is determined. The most plausible contour rule is the one chosen.

3.3. Dempster–Shafer–Talbot efficacy analysis

3.3.1. Description of the test case matrices

To illustrate the procedure outlined in the previous section, we have computed the matrix exponential of six square matrices from the MATLAB matrix gallery. These are:

Hanowa A $n \times n$ block matrix consisting of diagonal, sub-diagonal, and super-diagonal matrices.

- a $n \times n$ non-normal matrix where n is even
- the diagonal matrix is diagonal with constant value equal to -1
- the sub-diagonal matrix is diagonal with values $-1, -2, \ldots, -n/2$. The super-diagonal matrix is diagonal with values $1, 2, \ldots, n/2$
- eigenvalues $\{\lambda_j\}$ lie on a vertical line in the complex plane
- $\lambda_j = -1 \pm ji$ where $j = 1, 2, \ldots, n$.

Pei A symmetric matrix equal to the sum of the identity matrix and a constant matrix with elements all equal to 1.

- eigenvalues are on the real axis at $n + 1$ and 1
- eigenvalue at 1 has degeneracy $(n - 1)$.

Parter Toeplitz-diagonal constant matrix where $M_{i,j} = \frac{1}{i-j+0.5}$.

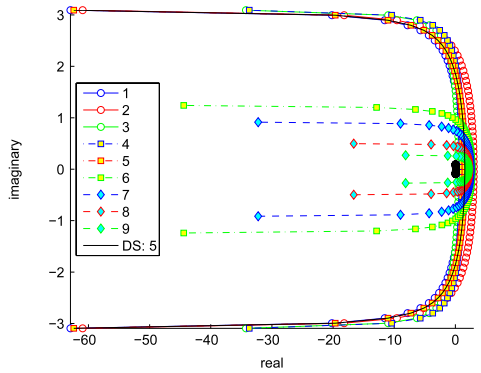
- eigenvalues have all positive real part and are symmetric about the real axis.

Riemann The $n \times n$ matrix $M = B_{(2:n+1,2:n+1)}$ where $B_{i,j} = \begin{cases} i-1 & \text{mod } (i,j) = 0 \\ -1 & \text{otherwise.} \end{cases}$

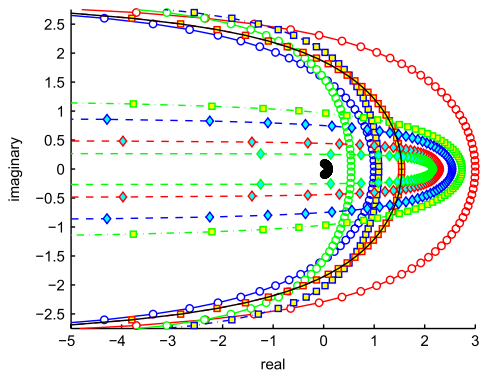
- a non-normal matrix
- eigenvalues have $|\lambda_j| < n/(n+1)$

Contour	pl	δ RE	Rank by pl
5	0.2247	0.0	1
1	0.2247	5.88e-9	2
2	0.2247	2.28e-7	3
3	0.15802	3.17e-5	5
4	0.15458	5.93e-5	6
6	0.19981	1.22e-2	4
7	0.15226	3.12e-1	7
8	0.13707	2.44e1	8
9	0.13631	9.36e6	9

(a) Tabulated error.



(b) Discretized Talbot contours.



(c) Eigenvalues and the Talbot contours.

Fig. 10. Parter 32 × 32 matrix for 64 contour points.

- $j \le \lambda_j \le j+1$
- all integers in the interval $((n+1)/3, (n+1)/2]$ are eigenvalues.

Tridiagonal The negative of the difference matrix where the diagonal equals 2 and the super- and sub-diagonals equal -1.

- eigenvalues are $2(1+\cos(\frac{k\pi}{n+1}))$ where $k = 1, 2, \ldots, n$
- eigenvalues are positive, real, and converge to zero as the matrix size increases.

Rando3 The random matrix M whose elements are $[-1, 0, 1]$ with equal probability.

- option three of the *gallery* random matrix
- has a different structure from the random matrix chosen for the training
- started with a random seed = 100
- eigenvalues are random.

The fact that the *Pei* and tridiagonal matrices are normal implies that the one and infinity norms of the matrices are the same. The other four matrices are non-normal. The eigenvalues of the matrices after scaling are plotted in Fig. 5. From it, one sees that this selection of test matrices has been chosen to provide examples with fundamentally different eigenvalue distributions. The values of the features for the scaled test matrices are provided in Table 5. Some of these values lie outside the bounds from the algorithm training so that not all features are used in the contour selection. Despite the limited training, the next section shows that the algorithm performs well.

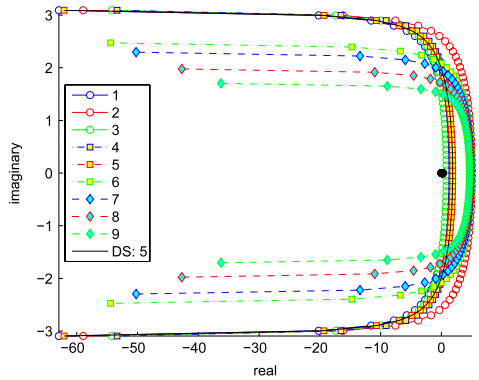
3.3.2. Analysis of the relative error per contour

The minimum relative error as computed by the DS Laplace transform algorithm is shown in Table 6. The contour corresponding to the minimum relative error is named in square brackets. The primary conclusion to draw from the table is that the DS-Talbot approach is accurate.

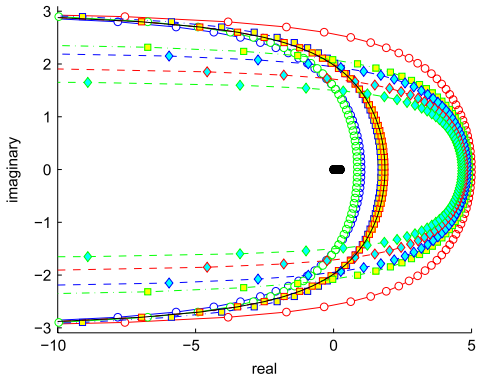
The reason for the exclusion of the 128 × 128 Riemann matrix is that it has feature values which are slightly outside of the boundaries of those obtained from the training matrices. The decision to define the mass to zero outside of the training set ranges was an arbitrary choice. In this case of the Riemann matrix, no masses where assigned and a contour could not be chosen. This illustrates that a major limiting factor to accuracy is not the Talbot contour integration but the size and diversity of the training set. More on this will be discussed in the concluding section on current and future research.

Contour	pl	δ AE	Rank by pl
7	0.46055	3.16e-2	7
6	0.52493	6.02e-3	4
4	0.47732	4.55e-7	5
5	0.55432	2.85e-4	1
2	0.55432	6.64e-8	3
1	0.55432	4.46e-8	2
8	0.45428	2.40e-1	8
9	0.45426	1.26e0	9
3	0.47451	2.94e-7	6

(a) Tabulated error.



(b) Discretized Talbot contours.



(c) Eigenvalues and the Talbot contours.

Fig. 11. Tridiagonal 32 × 32 matrix for 64 contour points.

The large relative errors reported in Table 6(f) for the tridiagonal matrix are due to the small matrix exponential element values. For increasing matrix size, the elements of the exponential approach the limits of double precision computations. Thus, the relative error can be large, even though the absolute difference between the MATLAB *expm* and Dempster–Shafer–Talbot approaches is quite small ($\sim 10^{-13}$ for all four matrix sizes $N = 16, 32, 64, 128$).

3.3.3. Comparison of the contour ranking from the minimum relative error and the Dempster–Shafer theory

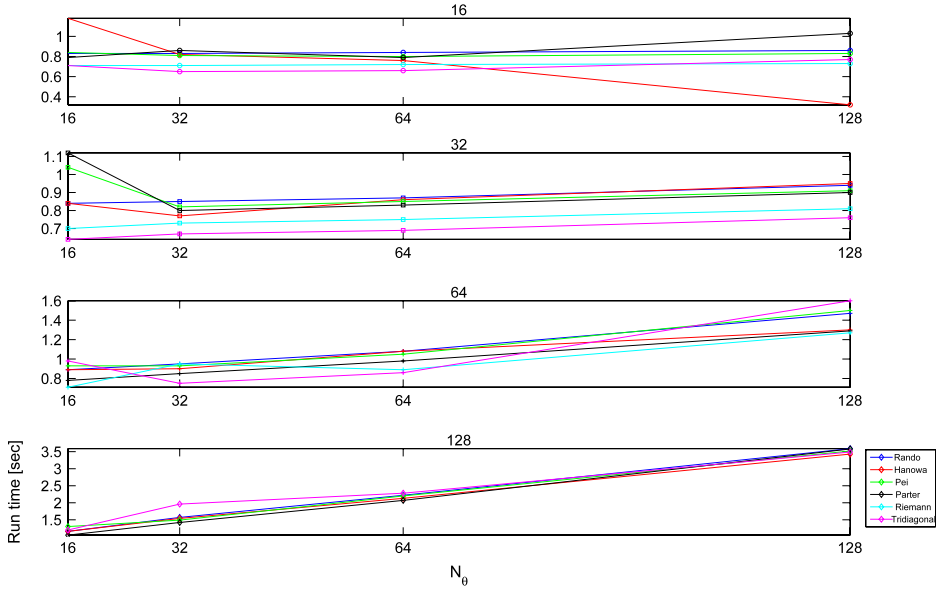
From the algorithm training, one finds that contour 5 results in the smallest relative errors for most matrices. Contours 1 and 2 also have relative errors on the same order as contour 5. Thus, the DS algorithm, correctly, chooses contour 5 for most cases. This is seen in Table 7 for the six test matrices. Again, the relative error results in the last table for the tridiagonal matrix are skewed due to the limited precision of the result from the *expm* function.

Another way to view the effectiveness of the Dempster–Shafer evidential reasoning is to show how it ranks the contours for the various test matrices. Focusing on the Rando3 matrix and integration with 64 Talbot contour points, one obtains the results in Table 8 for increasing matrix size. The ranking of the contour by relative error compares very well with ranking based on plausibility pl from the Dempster–Shafer reasoning. The good agreement between rankings is consistent with the fact that the training was performed using random matrices.

The contours for the random matrix of size 32 are shown in Fig. 6. For the other square 32 × 32 test matrices with 64 contour integration values, one obtains the results in Figs. 7–11. The figures show that the exact contour shapes change considerably from matrix to matrix. For smaller matrices, they also show that the truncated contours that extend further towards negative infinity correspond to less error. For all six test cases, contours 5, 1, and 2 corresponded to the lowest error while contours 8 and 9 performed poorly. From a practical implementation perspective, the fact that contour 5 corresponds to a relatively simple rule $[\sigma = \|A\|, \mu = 1, \nu = 1]$ is encouraging. Also, the scaling of the matrices prior to exponentiation is clearly beneficial as it brings the eigenvalues within the $\pm\pi i$ bounds that correspond to the contour from parameter rule 5 in Table 2.

3.3.4. Run time analysis

One final topic to address is the run time that is required to compute the matrix exponential using the Dempster–Shafer–Talbot calculation. Training the algorithm on the random matrices in Table 3 has required continuous computing



(a) Run times for the test matrices versus the # of contour points.

	16	32	64	128
16	0.83	0.84	0.89	1.16
32	0.83	0.85	0.95	1.57
64	0.84	0.87	1.08	2.22
128	0.86	0.94	1.47	3.59

(b) Rando3 exponentiation run times (sec): column-matrix size, rows-# contour points.

Fig. 12. Run time comparison.

on a single dedicated computer for a number of days. The majority of the run time is taken by the matrix inversions. The features and errors computed for each contour were stored and then quickly post-processed to generate the mass functions.

The actual employment of the DST algorithm on the test matrices required only a few seconds. Run times are summarized in Fig. 12(a). The values in the table for the Rando3 matrix are typical of the trends for all of the matrices. The run time grows linearly with the number of contour grid points and quadratically with the matrix size. The Dempster–Shafer–Talbot method is slower than the MATLAB rational approximation due to the need to perform a matrix inversion for each contour point. The MATLAB scripts however have not been fully optimized.

4. Conclusion

In this paper, we have presented a method for selecting an optimal rule for the parameters which define the complex contour in Talbot's method for the numerical inversion of the Laplace transform. This selection is performed using the Dempster–Shafer theory of evidential reasoning from decision science. The combined Dempster–Shafer–Talbot approach has been formulated for a general Laplace space function $F(s)$ in sufficient detail for replication. It has also been concretely demonstrated for the particular problem of estimating the exponential of a complex matrix through the inversion of the corresponding resolvent matrix. Main points from this demonstration have been the selection of features, the formulation of contour parameter rules, and the algorithm's training on random matrices. An analysis of the algorithm's efficacy reveals that the approach has merit.

With regards to further research, there are a number of possibilities that stem from the work in this paper. One is the application of the Dempster–Shafer theory to the selection between numerical inversion procedures themselves, such as Talbot's method, the Weeks method [20], and the Post formula [6]. Another clear avenue for future work is the application to specific real world problems with complicated expressions for $F(s)$. The adequate training of the algorithm for reliable inversion in these real world scenarios is an interesting problem. Related to this is the reduction of the time required for prior training. The use of the emerging graphics processing unit based acceleration [21] is a promising approach. The evaluation of $F(s)$ at the discrete points on the discretized Talbot contour clearly lends itself to the parallelization that is available on graphics processing units [22].

Acknowledgment

Patrick Kano would like to express his appreciation to Applied Energetics, Inc. for access to the company's computing resources when performing this research.

References

[1] A. Talbot, The accurate numerical inversion of Laplace transforms, J. Inst. Math. Appl. 23 (1979) 97–120.
[2] K. Sentz, S. Ferson, Combination of evidence in Dempster–Shafer theory, Sandia National Laboratory Report, SAND2002-0835, 2002.
[3] R. Bellman, Numerical Inversion of the Laplace Transform: Applications to Biology, Economics, Engineering, and Physics, American Elsevier Pub. Co., New York, 1966.
[4] J.A.C. Weideman, L.N. Trefethen, Parabolic and hyperbolic contours for computing the Bromwich integral, Math. Comp. 76 (2007) 1341–1356.
[5] J.A.C. Weideman, Algorithms for parameter selection in the Weeks method for inverting the Laplace transform, SIAM J. Sci. Comput. 21 (1) (1999) 111–128.
[6] P. Kano, M. Brio, Application of Post's formula to optical pulse propagation in dispersive media, Comput. Math. Appl. 59 (2) (2010) 629–650.
[7] D.G. Duffy, On the numerical inversion of Laplace transforms: comparison of three new methods on characteristic problems from applications, ACM Trans. Math. Software 19 (1993) 333–359.
[8] U. Graf, Applied Laplace Transforms and z-Transforms for Scientists and Engineers: A Computational Approach Using a Mathematica Package, Birkhäuser Verlag, Basel, 2004.
[9] A. Murli, M. Rizzardi, Algorithm 682. Talbot's method for the Laplace inversion problem, ACM Trans. Math. Software 16 (1990) 158–168.
[10] L.N. Trefethen, J.A.C. Weideman, T. Schmelzer, Talbot quadratures and rational approximations, BIT 46 (2006) 653–670.
[11] P.P. Valkó, J. Abate, Numerical inversion of Laplace transform with multiple precision using the complex domain, Mathematica Information Center: Mathsource, 2003, http://library.wolfram.com/infocenter/MathSource/5026/.
[12] J. Lancaster, S. Blackman, Joint IMM/MHT tracking and identification for multi-sensor ground target tracking, in: 9th International Conference on Information Fusion, 2006.
[13] S. Seims, A study of Dempster–Shafer theory of evidence in comparison to classical probability combination, California Polytechnic State Univeristy Master's Thesis, 2009.
[14] R. Wood, Cognitive systems: advances in processing capabilities will revolutionize system performance, Technology Today (3) (2005) 23. www.raytheon.com/technology_today/archive/2005_Issue3.pdf.
[15] R. Yager, L. Liu, Classic Works of the Dempster–Shafer Theory of Belief Functions, Springer, 2010.
[16] C. Moler, C. Van Loan, Nineteen dubious ways to compute the exponential of a matrix, twenty-five years later, SIAM Rev. 45 (1) (2003) 3–49.
[17] E. Eneyew, A. Weideman, Computation of the matrix exponential with application to linear parabolic PDEs, AIMS Postgraduate Diploma Thesis, 2010.
[18] J.A.C. Weideman, Optimizing Talbot's contours for the inversion of the Laplace transform, SIAM J. Numer. Anal. 44 (6) (2006) 2342–2362.
[19] A. Edelman, N. Raj Rao, Random matrix theory, Acta Numer. (2005) 1–65.
[20] P. Kano, M. Brio, J. Moloney, Application of Weeks method for the numerical inversion of the Laplace transform to the matrix exponential, Commun. Math. Sci. 3 (3) (2005) 335–372.
[21] J. Sanders, E. Kandrot, CUDA by Example: An Introduction to General-Purpose GPU Programming, Addison-Wesley, 2011.
[22] J. Cain, Graphics processing unit acceleration in MATLAB of a Dempster–Shafer approach for Talbot method contour selection, U. Arizona Honours Senior Thesis, 2011.